Java Backend Architecture

Interview Series

Chapter 9.6
DB Query Cache & Connection Pooling

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 9.6 – DB Query Cache & Connection Pooling

Introduction

Between the application cache tier and the raw database lies a critical infrastructure layer: **connection pooling** and **query-level caching**. Connection pooling reuses expensive TCP/SSL connections to the database, dramatically reducing latency per query. HikariCP is the de-facto Java connection pool — embedded in Spring Boot by default. For PostgreSQL at scale, PgBouncer offloads connection management from the DB process itself. Above the connection layer, Hibernate's second-level cache and query result cache reduce database round-trips by caching entities and query results in configurable providers (EhCache, Caffeine, Redis).

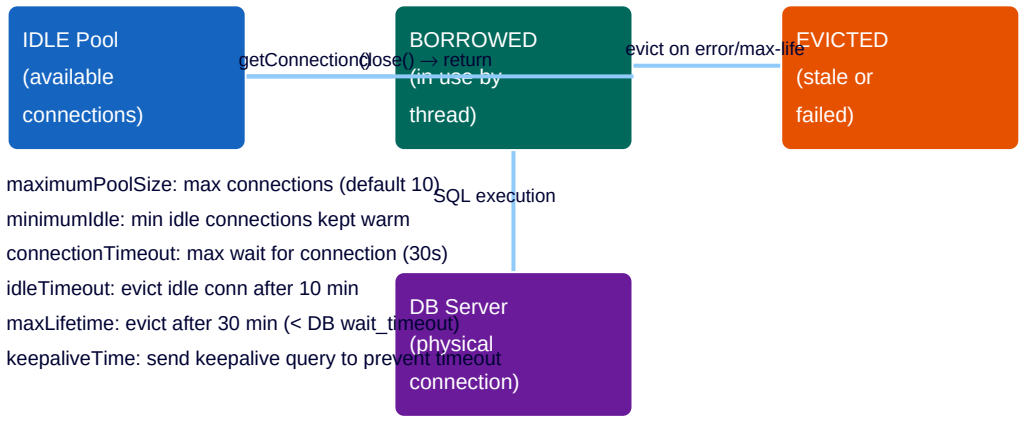
Core Concepts & Definitions

Connection Pool	Pre-established DB connections reused by threads; avoids TCP + auth handshake per query.
HikariCP	Ultra-fast JDBC connection pool; default in Spring Boot; byte-code proxy design.
maximumPoolSize	Max number of DB connections in the pool; set ≈ (cores * 2) + spindle count for OLTP.
connectionTimeout	Max time a thread waits for a pool connection before throwing SQLException (default 30s).
maxLifetime	Max age of a connection; must be < DB server wait_timeout to prevent dead connections.
PgBouncer	Lightweight connection pooler for PostgreSQL; sits between app and DB process.
Transaction Pooling	PgBouncer mode: DB connection held only during transaction; most efficient for OLTP.
Hibernate L1 Cache	Per-Session identity map; always enabled; avoids redundant queries within a Session.
Hibernate L2 Cache	SessionFactory-scoped shared entity cache; requires explicit config + provider (EhCache, Redis).
Query Cache	Hibernate cache for SQL result sets (entity IDs); invalidated on entity type modifications.
@Cache	JPA annotation to enable L2 caching for an entity; requires CacheConcurrencyStrategy.
CacheConcurrencyStrategy	READ_ONLY, NONSTRICT_READ_WRITE, READ_WRITE, TRANSACTIONAL — controls locking semantics.
Second-Level Cache Region	Named partition in L2 cache; each entity type gets its own region.
JDBC Batch	hibernate.jdbc.batch_size: group multiple INSERTs/UPDATEs into one JDBC batch statement.

Statement Cache	DB-side prepared statement cache; reduce parse overhead; controlled by JDBC driver params.

HikariCP Connection Pool

HikariCP Connection Pool Lifecycle



HikariCP: fastest Java JDBC connection pool; zero-overhead by design (byte-code proxy)

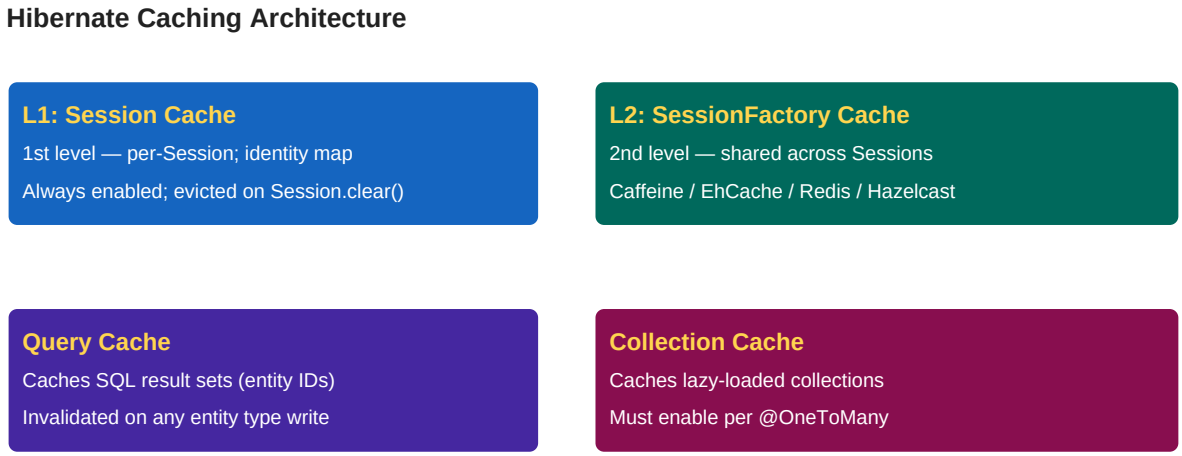
```
# application.yml — HikariCP configuration spring: datasource: url:
jdbc:postgresql://localhost:5432/mydb username: app_user password: secret hikari:
maximum-pool-size: 20 # max connections minimum-idle: 5 # warm idle connections
connection-timeout: 30000 # 30s wait for conn idle-timeout: 600000 # 10 min idle eviction
max-lifetime: 1800000 # 30 min max conn age keepalive-time: 60000 # 1 min keepalive ping
connection-test-query: SELECT 1 pool-name: MyHikariPool leak-detection-threshold: 5000 # warn
if conn held > 5s
```

```
// Programmatic HikariCP HikariConfig cfg = new HikariConfig();
cfg.setJdbcUrl("jdbc:postgresql://localhost:5432/mydb"); cfg.setUsername("app_user");
cfg.setPassword("secret"); cfg.setMaximumPoolSize(20); cfg.setMinimumIdle(5);
cfg.setConnectionTimeout(30_000); cfg.setMaxLifetime(1_800_000); cfg.setPoolName("MyPool");
cfg.addDataSourceProperty("cachePrepStmts", "true"); // prepared stmt cache
cfg.addDataSourceProperty("prepStmtCacheSize", "250");
cfg.addDataSourceProperty("prepStmtCacheSqlLimit", "2048"); HikariDataSource ds = new
HikariDataSource(cfg); // Metrics via Micrometer cfg.setMetricRegistry(meterRegistry); //
Exposes: hikaricp.connections, hikaricp.connections.acquire, hikaricp.connections.usage
```

HikariCP Pool Sizing Formula

The HikariCP author recommends: **pool_size = (core_count * 2) + effective_spindle_count**. For a 4-core SSD server: pool_size = 4*2 + 1 = 9. For PostgreSQL which forks a process per connection, keep the pool as small as possible while meeting throughput requirements. More connections = more RAM per pg backend + more lock contention. Test with pgbench.

Hibernate Second-Level Cache Architecture



Configuring Hibernate L2 Cache (EhCache)

```
<!-- pom.xml --> <dependency> <groupId>org.hibernate.orm</groupId>
<artifactId>hibernate-jcache</artifactId> </dependency> <dependency>
<groupId>org.ehcache</groupId> <artifactId>ehcache</artifactId>
<classifier>jakarta</classifier> </dependency>
```

```
# application.yml spring: jpa: properties: hibernate: cache: use_second_level_cache: true
use_query_cache: true region: factory_class:
org.hibernate.cache.jcache.internal.JCacheRegionFactory javax: cache: provider:
org.ehcache.jsr107.EhcacheCachingProvider uri: classpath:ehcache.xml jdbc: batch_size: 50
order_inserts: true order_updates: true
```

```
// Entity annotation @Entity @Cache(usage = CacheConcurrencyStrategy.READ_WRITE, region =
"products") public class Product { @Id private Long id; @Cache(usage =
CacheConcurrencyStrategy.READ_WRITE) @OneToMany(mappedBy = "product", fetch = FetchType.LAZY)
private List<Review> reviews; // ... } // Query cache List<Product> results = em.createQuery(
"SELECT p FROM Product p WHERE p.category = :cat", Product.class) .setParameter("cat",
"electronics") .setHint("org.hibernate.cacheable", true) // enable query cache
.setHint("org.hibernate.cacheRegion", "product-queries") .getResultList();
```

CacheConcurrencyStrategy Comparison

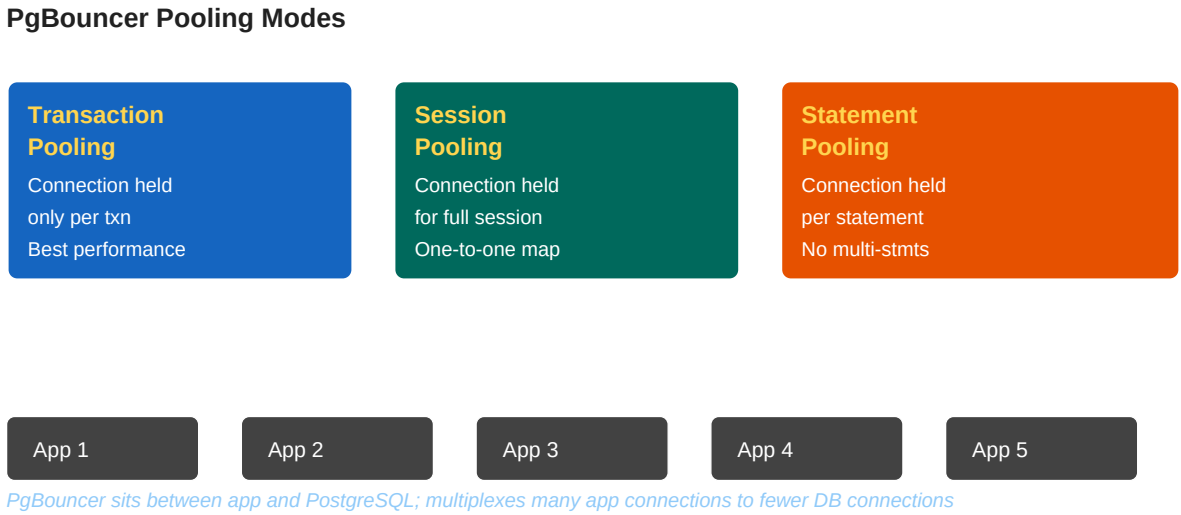
Strategy	Read	Write	Use Case
READ_ONLY	Fast (no lock)	Not supported	Immutable reference data
NONSTRICT_READ_WRITE	No lock	Invalidate (async)	Rarely updated, no strict consistency needed
READ_WRITE	Soft lock on read	Lock + update	Updated entities, eventual consistency OK
TRANSACTIONAL	JTA lock	XA transaction	Strict consistency, JTA required (EhCache + JT

Hibernate L2 Cache with Redis (via Spring Data Redis)

```
<!-- Alternative: use Redisson as Hibernate L2 provider --> <dependency>
<groupId>org.redisson</groupId> <artifactId>redisson-hibernate-6</artifactId>
<version>3.24.3</version> </dependency>
```

```
# application.yml — Redisson as Hibernate L2 provider spring: jpa: properties: hibernate: cache: use_second_level_cache: true region: factory_class: org.redisson.hibernate.RedissonRegionFactory redisson: config: redisson.yaml # redisson.yaml singleServerConfig: address: "redis://localhost:6379"
```

PgBouncer Connection Pooling



PgBouncer sits between app and PostgreSQL; multiplexes many app connections to fewer DB connections

```
# pgbouncer.ini [databases] mydb = host=pg-primary port=5432 dbname=mydb [pgbouncer] pool_mode = transaction # transaction pooling (recommended) max_client_conn = 10000 # total clients default_pool_size = 25 # per DB+user pair reserve_pool_size = 5 # extra for spikes max_db_connections = 100 # total DB connections server_idle_timeout = 600 # evict idle server conn listen_port = 6432 auth_type = md5 log_connections = 1 log_disconnections = 1
```

App connects to PgBouncer on port 6432 as if it were PostgreSQL. PgBouncer multiplexes connections to actual PostgreSQL on port 5432. In transaction mode, a PostgreSQL connection is held only for the duration of a transaction, then returned to pool — allowing thousands of application connections to share tens of PostgreSQL connections.

MyBatis L2 Cache

```
<!-- MyBatis mapper XML --> <mapper namespace="com.example.ProductMapper"> <!-- Enable L2 cache for this namespace --> <cache eviction="LRU" flushInterval="60000" size="512" readOnly="true"/> <select id="findById" resultType="Product" useCache="true"> SELECT * FROM products WHERE id = #{id} </select> <!-- Disable cache for volatile queries --> <select id="findActiveOrders" resultType="Order" useCache="false"> SELECT * FROM orders WHERE status = 'PENDING' </select> </mapper>
```

JDBC Statement Cache (PreparedStatement Cache)

```
// PostgreSQL JDBC - server-side prepared statement cache Properties props = new Properties(); props.setProperty("prepareThreshold", "5"); // prepare after 5 uses props.setProperty("preparedStatementCacheQueries", "256"); // LRU cache size // MySQL JDBC - client-side prepared statement cache String url = "jdbc:mysql://localhost:3306/mydb" + "?cachePrepStmts=true" + "&prepStmtCacheSize=250" + "&prepStmtCacheSqlLimit=2048" + "&useServerPrepStmts=true"; // HikariCP passes these as dataSourceProperties: hikariConfig.addDataSourceProperty("cachePrepStmts", "true"); hikariConfig.addDataSourceProperty("prepStmtCacheSize", "250");
```

Interview Questions & Answers

Q1. Why is connection pooling essential for Java applications?

Creating a DB connection involves TCP handshake, authentication, and protocol negotiation — typically 5-50ms. Connection pooling pre-creates connections and reuses them, reducing per-query overhead to sub-millisecond pool acquisition. Without pooling, a 1ms query becomes a 50ms operation.

Q2. What is HikariCP and why is it the default in Spring Boot?

HikariCP is an ultra-fast JDBC connection pool using byte-code instrumentation (not reflection-based proxies) and a lock-free ConcurrentBag for connection management. It has zero-overhead design: getConnection() has sub-microsecond overhead. Spring Boot auto-configures it when hikari jar is on classpath.

Q3. What is the maximumPoolSize and how should it be set?

maximumPoolSize is the max number of DB connections in the pool. HikariCP author recommends (core_count * 2) + spindle_count. For a 4-core server: ~9. Larger is not always better: more connections = more DB memory, more lock contention. Test with realistic load and measure throughput vs latency.

Q4. What happens when the connection pool is exhausted?

Threads wait up to connectionTimeout (default 30s) for an available connection. If none available, SQLTransientConnectionException is thrown. Solutions: increase pool size, reduce query time, use connection leak detection (leakDetectionThreshold), or scale horizontally.

Q5. What is maxLifetime and why must it be less than DB server timeout?

maxLifetime is the max age of a pooled connection. After this time, HikariCP evicts and replaces it. It must be set LESS than the DB server's connection timeout (e.g., MySQL wait_timeout, PostgreSQL tcp_keepalives_idle). If DB closes a connection that HikariCP thinks is valid, the next query will fail with a stale connection error.

Q6. What is connection leak detection in HikariCP?

leakDetectionThreshold: if a connection is checked out for longer than this duration, HikariCP logs a warning with a stack trace of the acquiring thread. Helps find code that holds connections without releasing them. Set to 5000ms for most applications. Does not close the connection, only warns.

Q7. What is Hibernate's first-level (L1) cache?

Session-scoped identity map: within a Session, the same entity (by PK) is loaded only once. Subsequent findById calls return the cached instance. Always enabled, cannot be disabled. Cleared on session.clear() or session.evict(entity). Prevents duplicate queries within a transaction.

Q8. What is Hibernate's second-level (L2) cache?

SessionFactory-scoped shared cache across Sessions and transactions. Requires explicit config + provider (EhCache, Caffeine, Redis via Redisson/Hazelcast). Annotate entities with @Cache. Invalidated on entity updates within the same SessionFactory. Read-only entities get the best performance.

Q9. What are the CacheConcurrencyStrategy options in Hibernate?

READ_ONLY: no locking, best performance, for immutable data. NONSTRICT_READ_WRITE: no locking, async invalidation, slight staleness possible. READ_WRITE: soft locks on write, suitable for most mutable entities. TRANSACTIONAL: full JTA transaction, strictest, requires JTA provider.

Q10. What is the Hibernate Query Cache?

Caches the result sets of HQL/JPQL/Criteria queries (as lists of entity IDs + scalar values). Enabled per query: setHint('org.hibernate.cacheable', true). Cache is INVALIDATED whenever any entity of the queried type is updated — even if unrelated to query params. High invalidation rate makes it suitable only for stable reference data queries.

Q11. Why can the Hibernate Query Cache hurt performance?

The query cache stores entity IDs; those IDs are then fetched from L2 entity cache or DB — N queries for N results (N+1 problem if entities not in L2). Also, any write to the table invalidates ALL cached queries for that entity type. High-write tables make query cache counterproductive.

Q12. What is PgBouncer and what problem does it solve?

PgBouncer is a connection pooler for PostgreSQL. PostgreSQL forks a process per connection — thousands of connections eat GB of RAM. PgBouncer sits in front, accepts thousands of app connections, but maintains only tens of DB connections. Solves the PostgreSQL per-connection process overhead.

Q13. What are the three PgBouncer pooling modes?

Session pooling: one DB connection per client session — effectively no multiplexing. Transaction pooling: DB connection released after transaction commit/rollback — best multiplexing. Statement pooling: connection released after each statement — disallows multi-statement transactions.

Q14. Why is transaction pooling the preferred PgBouncer mode?

Transaction pooling allows the most connection multiplexing. In OLTP, transactions are short, so DB connections are returned quickly. 1000 app connections can be served by 25 DB connections. Limitation: SET commands, advisory locks, LISTEN/NOTIFY, and prepared statements per-session are not preserved across transactions — must disable per-session features.

Q15. What is the difference between HikariCP and PgBouncer?

HikariCP pools connections inside the Java application process — reduces TCP overhead from app to DB. PgBouncer pools at the DB process level — reduces PostgreSQL process count. They are complementary: PgBouncer handles many application instances; HikariCP handles many threads within one instance.

Q16. What is the N+1 query problem and how does caching help?

N+1: loading N parent entities, then issuing 1 query per parent to load child collection = N+1 DB queries. Hibernate L2 cache can serve child entities from cache on second access. Better solution: JOIN FETCH or @BatchSize (batch child loads). @Cache on collection also helps for repeated access patterns.

Q17. How do you enable batch inserts in Hibernate?

Set hibernate.jdbc.batch_size=50, hibernate.order_inserts=true, hibernate.order_updates=true. Use Spring Data JPA saveAll() or EntityManager.persist() in a loop — Hibernate batches 50 INSERTs per JDBC batch call. Use @GeneratedValue with SEQUENCE (not IDENTITY) — IDENTITY breaks batching.

Q18. What is the Spring @Transactional interaction with Hibernate L2 cache?

Within a transaction, Hibernate uses L1 cache (Session). Commits write to L2 cache and invalidate stale regions. With READ_WRITE strategy, soft locks prevent dirty reads from L2 during ongoing transactions. Rollbacks don't populate L2 cache. Multi-tenant scenarios require cache region isolation.

Q19. What is a JDBC connection leak and how do you detect it?

A connection leak occurs when code borrows a connection but never returns it (e.g., exception path without try-with-resources). Pool grows to maximum, then new requests timeout. Detect: HikariCP leakDetectionThreshold logs stack trace of leaking acquisition. Fix: always use try-with-resources for Connection, PreparedStatement, ResultSet.

Q20. How do you configure read replicas with HikariCP?

Create two DataSource beans — one for primary (write), one for read replica. Use AbstractRoutingDataSource to route queries based on transaction read-only flag: @Transactional(readOnly=true) routes to replica. Or use Spring's TransactionRoutingDataSource.

Alternatively, use ProxySQL/PgPool-II for transparent routing.

Q21. What is the prepared statement cache in JDBC drivers?

JDBC drivers maintain a client-side LRU cache of PreparedStatement objects (SQL + parse plan). Avoids re-parsing the same SQL on the DB server. PostgreSQL additionally does server-side prepared statements after prepareThreshold uses. Enable: cachePrepStmts=true, prepStmtCacheSize=250 in connection URL.

Q22. What is the MyBatis L2 cache and its limitations?

MyBatis L2 cache stores query results per Mapper namespace. Enabled with in mapper XML. Eviction: LRU, FIFO, SOFT, WEAK. Limitation: cache is invalidated on any INSERT/UPDATE/DELETE in the namespace, not per-entity. Not distributed — per-JVM. For production, replace with Caffeine or Redis via custom Cache implementation.

Q23. What is connection validation in HikariCP?

HikariCP validates connections before lending them to threads. connectionTestQuery (SELECT 1) is used for JDBC 4- drivers. JDBC 4+ uses Connection.isValid(). keepaliveTime sends a validation query periodically to prevent firewall/DB from closing idle connections. validationTimeout is max time for validation (default 5s).

Q24. How do you monitor HikariCP pool health in production?

Micrometer metrics (with HikariCP 3.4+): hikaricp.connections (total), hikaricp.connections.idle, hikaricp.connections.active, hikaricp.connections.pending (threads waiting), hikaricp.connections.acquire (latency), hikaricp.connections.usage (hold time). Alert: pending > 0 sustained means pool exhausted.

Q25. What is the 'connection storm' problem and how do you prevent it?

On application startup, all pool connections are created simultaneously, overwhelming the DB. HikariCP uses minimumIdle and lazy initialization. Set minimumIdle=2 to avoid cold-start storm. Or use Spring's DataSourceInitializer to pre-warm gradually. PgBouncer server_login_retry adds delay.

Q26. Explain the Hibernate EntityManager.find() caching behavior.

find() by PK: 1) Check L1 (Session) — return if found. 2) Check L2 — return if found, populate L1. 3) Issue SELECT — populate L1 + L2 based on @Cache annotation. JPQL queries bypass L1/L2 unless query cache is enabled and entity is also in L2.

Q27. What is the difference between Hibernate L2 cache regions?

Each entity type has its own L2 region (e.g., 'com.example.Product'). Collection associations have their own region (e.g., 'com.example.Product.reviews'). Query cache uses dedicated 'query-cache' region. Regions allow per-entity cache configuration (TTL, size) and independent invalidation.

Q28. How do you evict specific entities from Hibernate L2 cache?

Via Cache API: entityManager.getEntityManagerFactory().getCache().evict(Product.class, id); or evictAll(Product.class). Via HibernateSessionFactory: sessionFactory.getCache().evictEntity(Product.class, id). In Spring: inject JpaVendorAdapter and cast to HibernateJpaVendorAdapter to get SessionFactory.

Q29. What is ProxySQL and how does it compare to PgBouncer?

ProxySQL is a MySQL-specific proxy that provides connection pooling, query routing (read/write splitting), query rewriting, query caching (query result cache), and failover. PgBouncer is PostgreSQL-specific, simpler, pure connection pooler with no query awareness. ProxySQL competes with MySQL Router; PgBouncer competes with pgpool-II.

Q30. What are the best practices for HikariCP in containerized environments?

1) Set maximumPoolSize based on DB capacity, not pod count. 2) Use Kubernetes liveness probe that checks pool health. 3) Set maxLifetime < DB server connection timeout. 4) Configure keepaliveTime to

survive network load balancer idle connection drops (typically 350-600s). 5) Use environment variables for pool sizing to allow per-environment tuning without code changes.

Q31. How does Hibernate statistics work and what metrics are useful?

Enable: hibernate.generate_statistics=true. Expose via SessionFactory.getStatistics(). Key metrics: SecondLevelCacheHitCount, SecondLevelCacheMissCount, SecondLevelCachePutCount, QueryCacheHitCount, EntityLoadCount, CollectionLoadCount. Spring Actuator: /actuator/metrics/hibernate.second.level.cache.requests with hit/miss tags.

Q32. What is the 'connection wait' vs 'connection acquisition' latency in HikariCP?

Connection acquisition time (hikaricp.connections.acquire) = time from getConnection() call to receiving a connection from pool. If pool has idle connections: sub-microsecond. If pool is exhausted: wait time until connection freed. High acquisition latency indicates pool too small or queries too slow.

Q33. Describe a full database caching architecture from connection to query result.

1) PgBouncer (PostgreSQL only): multiplexes app connections to fewer DB connections. 2) HikariCP: per-JVM connection pool; JDBC statement cache. 3) Hibernate L1: per-Session identity map. 4) Hibernate L2: per-SessionFactory entity/collection cache (EhCache/Redis). 5) Hibernate Query Cache: query result cache for stable queries. 6) Application cache: @Cacheable with Caffeine/Redis over service methods. Each layer reduces load on the next.